MEDICONSULT / FINANCE PLATFORM

Project guide

Architecture, endpoints & workflows

A source-based reference for the frontend application.

134 API method/path pairs. 62 declared routes.

Accounting, transactions, banking and reporting.

---

14 October 2026

Repository: mediconsult-finance

Source baseline: 91ba6a8 + current working tree

PRIMARY COLOR #236167 / ARABIC RTL APPLICATION

Inside this guide

A practical reference for developers, reviewers, and project stakeholders. The endpoint inventory is extracted from the current JavaScript source, preserving path casing and HTTP methods.

01 / Project at a glance 3

02 / Architecture & data flow 4

03 / Codebase map & development 5

04 / Authentication & shared UI flow 7

05 / Transaction workflows 8

06 / Banking & reconciliation workflows 9

07 / Reporting & integration contracts 10

08 / Browser route catalog 12

09 / API endpoint catalog 15

10 / Form field inventory & rules 28

11 / Source references & maintenance 31

Reading the scope
This guide describes the frontend calls and UI flows found in this repository. It does not claim a complete backend API specification. Exact backend response schemas, status codes, permissions, and database behavior are not supplied here.

01 / Project at a glance

The application brings financial operations into one Arabic interface: master data and account setup feed transaction forms, bank operations support payments and reconciliation, and report pages expose financial views and Excel downloads.

Area	Implemented scope
Accounting	Account tree, roots and children, account search, create/update forms, cost center tree/search, and account-to-cost-center linking.
Master data	Customer list/detail; supplier list/detail, lookup filters and finance-information updates.
Transactions	Journal list/create/detail/edit actions; customer, supplier and batch invoices; cash vouchers; cash movements; receivable/payable commercial papers.
Banking	Bank and bank-account CRUD, transaction lists, transfer creation/approval/cancellation, cheque management/attachments/status updates, reconciliation item actions and finalization.
Reporting	General ledger and party statements, aging, trial balance, income statement, balance sheet, and nine banking report API modules including cheque summary.
Authentication	Login via API, Redux credentials, local storage persistence and the authenticated main layout.

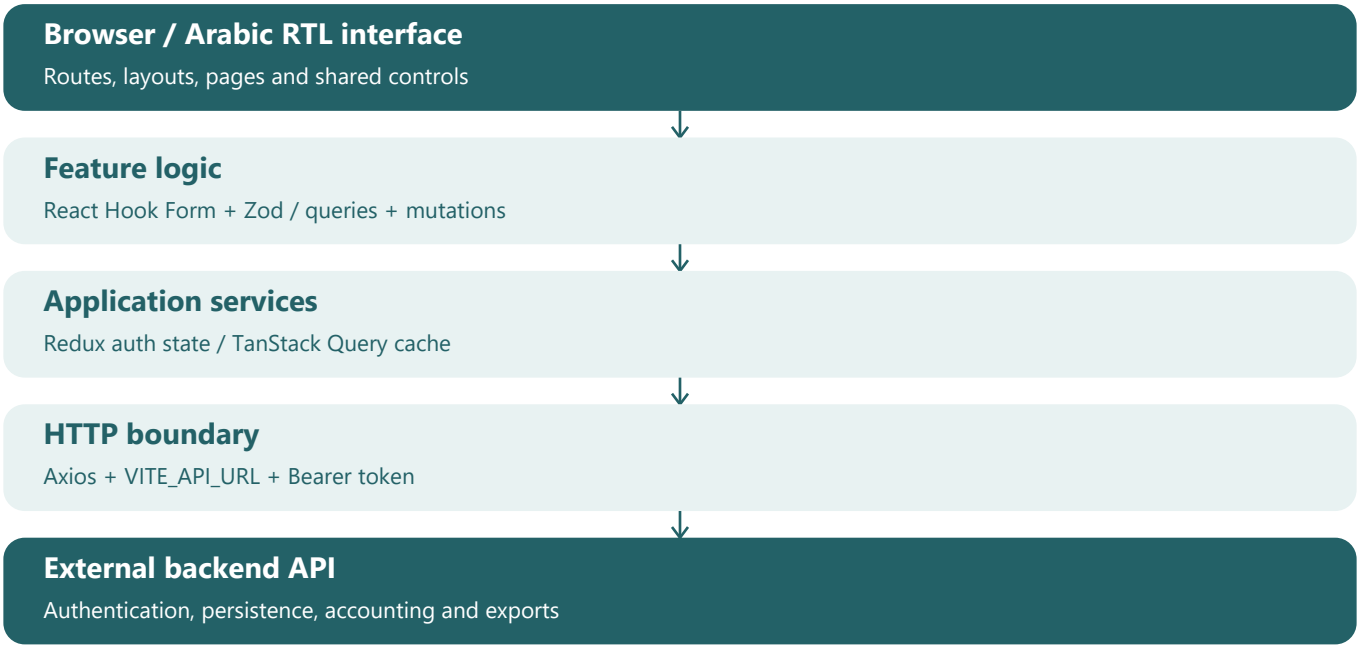
Dashboard data is illustrative
The home page explicitly defines sample statistics and recent activity in code. No dashboard API module is present. Those figures should not be interpreted as live financial totals.

Repository facts

The inspected source contains 340 JavaScript/JSX files, 62 explicit path declarations, and 147 resolved API references representing 134 unique HTTP method/path pairs. Duplicate wrappers are retained in the API catalog by source module.

Design tokens from src/index.css: primary/main #236167; body #f7fafc; borders #e1e1e4; secondary #d2d6da; dark #0b0c0c. The application uses the Cairo font and declares RTL direction in index.html. This English guide uses Segoe UI for compact technical documentation.

02 / Architecture & data flow



Boot and route composition

index.html loads src/main.jsx. The entry point mounts StrictMode, the Redux Provider, QueryClientProvider, ToastProvider and RouterProvider. The central router composes feature route arrays. /auth hosts login; / hosts the main layout and dashboard index. The main layout redirects to /auth when the Redux token is absent.

State ownership

Layer	Responsibility
Redux	store.js combines main and auth reducers. The main slice controls shared UI state; auth holds the token and user.
TanStack Query	Feature hooks own server data and mutations. QueryClient defaults: staleTime = 60 seconds; retry = 1.
Local form state	React Hook Form tracks inputs; Zod validates forms; helpers map UI strings and dates into request payloads.
Browser storage	token and user are persisted in localStorage. Axios reads token for each request.
Routing and loading	lazyPage uses React.lazy and Suspense with PageLoader. The top-level route supplies RouteErrorPage.

Read / write cycle

Read: route page -> feature query hook -> feature API function -> Axios -> backend -> response normalization -> query cache -> UI. Write: form validation -> payload mapping -> mutation -> API request -> success/error feedback -> affected query invalidation -> refreshed UI. Invalidation scope varies by feature; it is not a universal refresh of all reports.

03 / Codebase map & development

Location	Purpose
`src/main.jsx`	Mounted providers and router; the real runtime entry point.
`src/index.css`	Global CSS, Tailwind import, theme colors and Cairo typography.
`src/app/api/`	axiosInstance.js and queryClient.js: shared HTTP/caching configuration.
`src/app/layouts/`	Authentication and main layouts; authentication gate and navigation shell.
`src/app/routes/router.jsx`	Composition of authentication, dashboard and feature routes.
`src/app/store/store.js`	Redux reducer composition.
`src/app/assets/`	Bundled logos and background images.
`src/app/App.jsx`	Empty placeholder component; it is not mounted by main.jsx.
`src/features/`	accounting, auth, banking, dashboard, master-data, reports and transactions.
`src/shared/`	components, UI controls, lib helpers, shared store and formatting/storage utilities.
`public/`	Assets served directly by Vite.
`dist/`	Generated production build, ignored by Git.

Feature conventions

Features generally separate api, hooks, pages, components, routes, validation and utils. Query-key files centralize cache identifiers where present. Invoice variants reuse transactions/invoices/shared. Account tree code currently lives under accounting/tree/accouts-tree (the spelling is preserved from the repository).

Declared dependencies

Package	Declared version	Role
react	^19.2.0	UI components
react-dom	^19.2.0	Browser rendering
react-router-dom	^7.13.0	Routing
@tanstack/react-query	^5.90.20	Server state and mutations
@reduxjs/toolkit	^2.11.2	Application state
react-redux	^9.2.0	Redux React bindings
axios	^1.13.3	HTTP client
react-hook-form	^7.71.1	Form state
@hookform/resolvers	^5.2.2	Validation bridge
zod	^4.3.6	Validation schemas
tailwindcss	^4.1.18	Utility CSS
@tailwindcss/vite	^4.1.18	CSS build integration
lucide-react	^0.563.0	Icons

Build tools: vite is aliased to rolldown-vite@7.2.5; @vitejs/plugin-react-swc ^4.2.2. Tooling includes ESLint ^9.39.1/^9.39.2 and Prettier ^3.8.1. package-lock.json records resolved versions; ranges above are from package.json.

Runtime and scripts

Node requirement from the installed Vite package: ^20.19.0 or >=22.12.0. Install with npm ci. Configure VITE_API_URL in a root .env file, then start npm run dev. Valid backend credentials are needed for authenticated pages.

Command	Purpose
npm ci	Install locked dependencies.
npm run dev	Development server with hot reload.
npm run build	Production bundle in dist/.
npm run preview	Local preview of the built bundle.
npm run lint	ESLint checks.
npm run format	Prettier formatting across the repository.

There is no npm test script. Existing tests are Node test-runner files in banking/banks/validation/bank.validation.test.js and banking/cheques/utils/batch-invoice-summaries.test.js. Run them with node --test followed by those two file paths. This guide was checked for source inventory and PDF layout; it does not report a new application test run.

Build and deployment

Vite uses @vitejs/plugin-react-swc and @tailwindcss/vite. The vite dependency is aliased to rolldown-vite@7.2.5. Set VITE_API_URL during the build; VITE_ values are visible in browser code. Deploy dist/ to a static host. vercel.json rewrites /(.*) to /index.html for browser-route fallback. Other hosts need an equivalent fallback.

04 / Authentication & shared UI flow

Login, session restoration and logout

01	**Enter credentials** Login form supplies username and password to POST /Auth/login.
02	**Persist the session** useLogin reads token, username and fullName from the Axios response data. It stores token/user in localStorage and dispatches setCredentials.
03	**Enter the main layout** RootLayout checks the Redux auth token. A missing token produces a redirect to /auth; a present token allows the navigation shell and child route.
04	**Authorize requests** The Axios request interceptor reads localStorage.token and adds Authorization: Bearer <token>.
05	**Logout** Navbar removes token and user from storage and dispatches clearCredentials. The layout then redirects through its token check.

Source: auth/api/auh-api.js; auth/hooks/useLogin.js; auth/store/auth-slice.js; app/layouts/root-layout.jsx; shared/components/navbar.jsx

Frontend authentication boundary
The layout gate checks token presence. Server authorization, token lifetime and role policies must be verified against the backend. The shared Axios file contains a request interceptor; it does not define a token-refresh or global 401 response interceptor.

Common page interactions

Shared UI supplies tables, form controls, searchable selects, filters, modals, pagination and loaders. Navigation labels and loading messages are Arabic. ToastProvider delivers success/error messages. Reusable search components and debounce helpers support account, cost center and party selection.

05 / Transaction workflows

Journal entry lifecycle

01	**Load context** Fetch accounts/cost centers, party/invoice context, financial periods and statuses through feature hooks and lookup API functions.
02	**Validate the entry** Require entry date, journal type, financial period, status and at least one detail. Each detail needs an account and a debit or credit amount. Total debit must equal total credit and must be nonzero.
03	**Map and save** buildJournalEntryPayload converts IDs/amounts and dates. Manual creation sets sourceType = Manual and sourceId = 1. POST /journal-entries creates; PUT /journal-entries/{id} updates.
04	**Post or reverse** Explicit actions call POST /journal-entries/{id}/post with postedBy or /reverse with reversedBy. Backend rules determine whether the action is permitted.

Source: transactions/entries/validation/journal-entry.validation.js; utils/journal-entry.utils.jsx; api/entries.api.js

Invoice creation, payment and batches

01	**Choose the invoice view** GET /invoices applies showOnlyCustomersInvoices, showOnlySuppliersInvoices or hasBatchOnly for the selected view.
02	**Build invoice details** Load types, parties, products/services, periods, statuses and next number. Validate dates, party selection and at least one line; quantity is at least 1 and price/discount/tax values are nonnegative.
03	**Calculate and save** Gross = sum(quantity x unitPrice). Discounts include line percentages plus header discount. Net = max(gross - discounts + header tax, 0). POST /invoices creates; PUT /invoices/{id} updates.
04	**Pay an invoice** POST /invoices/{id}/pay receives amount, payment date and Check, Cash or BankTransfer. Required account/bank fields depend on the selected mode.
05	**Process a batch** GET /batches/{batchNumber} loads a batch. POST /batches/create-invoice creates its invoice. POST /batches/{batchNumber}/close is the batch-closing action; its mutation invalidates invoice, batch and journal caches.

Source: transactions/invoices/shared/api/invoices-api.js; utils/invoice-form-utils.js; utils/pay-invoice-utils.js; hooks/invoices.mutations.js

Cash vouchers, cash movements and commercial papers

Cash vouchers have list/detail/create/update and status lookup calls, plus invoice detail lookup by invoice number. Cash transactions expose balance, paged list, detail, create/update and deletion with a reason query parameter. Commercial-paper creation chooses notes-receivable or notes-payable based on RECEIVABLE/PAYABLE; unsupported paper types throw before making the request. List/detail/update/delete share /commercial-papers paths.

06 / Banking & reconciliation workflows

Bank transfer workflow

01	**Choose source and destination** In a bank detail page, select the source account. Internal transfers require destination bank/account; external transfers require a party. Amount must be positive.
02	**Review the transfer** The form builds a payload and opens its confirmation stage before calling createTransfer.
03	**Create and inspect** POST /bank-transfers submits the confirmed payload. The UI resets and opens the transfer context when an ID is returned.
04	**Approve or cancel** POST /bank-transfers/{id}/approve approves. POST /bank-transfers/{id}/cancel sends cancelReason. Mutation hooks refresh transfer lists/details.

Source: banking/banks/components/bank-transfer-form.jsx; validation/bank-transfer.validation.js; api/banks.api.js

Cheque and batch invoice selection

01	**Select party and bank context** Cheque type controls customer/supplier requirements. The form validates numeric cheque number, positive amount, bank/account and chronological dates.
02	**Select batch invoices** GET /batches/{batchNumber}/invoice-summary returns supplier, invoice identifiers, invoice number and net amount. Duplicate invoices are blocked; selected invoices must belong to the same supplier.
03	**Create or update** POST /cheques creates; PUT /cheques/{id} updates. Batch totals are summed using rounded cents. Status changes use PATCH /cheques/{id}/status.
04	**Attach supporting material** POST /cheques/{id}/attachment uploads multipart/form-data with the field named Attachment.

Source: banking/cheques/api/cheques.api.js; validation/cheque.validation.js; utils/batch-invoice-summaries.js

Reconciliation workflow

01	**Create and load** POST /bank-reconciliations creates a reconciliation; GET its detail and /items loads the work list.
02	**Match statement items** POST .../items/{itemId}/match sends the full bankTransactionIDs selection. The frontend converts to positive finite numbers, removes duplicates and rejects an empty selection.
03	**Revise selections** Unmatch removes a match; ignore sends notes for an item. The API comment describes matching as replacement of the prior selection.
04	**Finalize** POST /bank-reconciliations/{id}/finalize completes the explicit finalization action. /reconcile remains a legacy compatibility call.

Source: banking/reconciliations/api/bank-reconciliations.api.js; hooks/bank-reconciliations.mutations.js

07 / Reporting & integration contracts

Report interaction flow

Choose filters -> build/clean query parameters -> GET report data -> display totals and rows -> request the corresponding export-excel endpoint -> receive a Blob -> create a browser object URL -> trigger an .xlsx download -> revoke the URL. Export hooks keep a local isExporting state and show success/error feedback.

Report	Frontend parameter behavior
General ledger	accountId, fromDate, toDate, financialPeriodId, costCenterId, customerId, supplierId, currencyId, partyType and sourceType. IDs become numbers; dates use the journal date helper; empty values are omitted.
Aging	referenceDate, partyType, customerId/supplierId, category, providerClass, importanceLevel, partyStatus, governorateId, sortBy, sortOrder, pageNumber and pageSize. Defaults differ between filter state (10) and query fallback (50) for page size.
Trial balance	financialPeriodId, fromDate, toDate, costCenterId and filterByCostCenter. includeOpeningBalance is forced to true. Roots and children support hierarchical browsing.
Income statement	fromDate, toDate, financialPeriodId and costCenterId. postedOnly and includeHeaderAccounts default to true, then caller params are spread in.
Balance sheet	asOfDate, financialPeriodId, costCenterId and filterByCostCenter. includeHeaderAccounts defaults to true.
Banking reports	Filters are passed through a cleanParams helper that excludes empty strings, null and undefined. All report export functions request responseType: blob.

Request conventions

All catalog paths are relative to VITE_API_URL. Placeholder braces represent values interpolated by the frontend; they are not literal braces in HTTP requests. Path casing is preserved, including /Auth/login, /Banks, /BankAccounts and /AccountCostCenterLink. Most writes send JSON. Cheque attachment is the multipart exception. URL encoding is explicit for batch close, batch invoice summary and cheque attachment identifiers; other interpolations follow the source as written.

Response normalization

There is no single response-envelope assumption. Banking extraction accepts arrays and nested data/items/list/$values/records/rows/result. Invoice normalization accepts response.data.invoice, response.invoice, response.data or the raw response; success:false is treated as failure. Several lists build pagination metadata with defaults. Trial balance expects accounts and supplies empty totals when its shape is absent. See the API module referenced with each endpoint for the precise return mapping.

Selected payload details

Action	Payload / conversion
`Invoice payment`	amount becomes a number; paymentMode becomes Check, Cash or BankTransfer; paymentDate gains T00:00:00. referenceNumber and irrelevant account fields become null.
`Transfer creation`	frombankId is a string; FromBankAccountID and amount are numbers. Internal transfers add tobankId and ToBankAccountID; external transfers add partyType and partyID. Preserve these exact field names.
`Reconciliation match`	{ bankTransactionIDs: ids } with the complete normalized selection.
`Journal actions`	Post sends { postedBy }; reverse sends { reversedBy }.
`Transfer cancellation`	{ cancelReason } with an empty-string default.
`Cheque attachment`	FormData field Attachment contains the file.
`Cash transaction deletion`	reason is supplied in the URL query configuration.

08 / Browser route catalog

The following entries come from route source files. Relative top-level feature paths are shown with a leading slash for readability. / is the dashboard index and /auth hosts the login index. These are UI routes, separate from the API paths in the next chapter.

Accounting

Browser path	Screen module
/accounts-tree	../pages/AccountsTree
/accounts-tree/:id	../pages/update-account
/accounts-tree/new	../pages/new-account
/cost-tree	../pages/cost-center-tree
/cost-tree/:id	../pages/update-cost
/cost-tree/new	../pages/new-cost
/link	../pages/link-account-cost-center

Banking

Browser path	Screen module
/banks	../pages/banks-page
/banks/new	../pages/new-bank
/banks/:id	../pages/bank-details
/banks/:id/*	../pages/bank-details
/cheques	../pages/cheques-page
/cheques/new	../pages/new-cheque
/cheques/:id	../pages/cheque-details
/reconciliations/new	../pages/new-reconciliation-page
/reconciliations	../pages/reconciliations-page
/reconciliations/:id	../pages/reconciliation-details-page

Master Data

Browser path	Screen module
/customers	../pages/customers-page
/customers/:id	../pages/customer-details
/suppliers	../pages/suppliers-page
/suppliers/:id	../pages/supplier-details

Reports

Browser path	Screen module
/aging-report	../pages/aging-report-page
/general-ledger	../pages/general-ledger-page
/general-ledger/supplier	../pages/supplier-account-statement
/general-ledger/customer	../pages/customer-account-statement
/bank-balances	../pages/bank-balances-page
/cheques-report	../pages/bank-cheques-report-page
/bank-reconciliation	../pages/bank-reconciliation-page
/bank-statement	../pages/bank-statement-page
/bank-transactions	../pages/bank-transactions-report-page
/bank-transfers	../pages/bank-transfers-report-page
/cash-flow	../pages/cash-flow-page
/outstanding-cheques	../pages/outstanding-cheques-page
/balance-sheet	../pages/balance-sheet-page
/income-statement	../pages/income-statement-page
/trial-balance	../pages/trial-balance-page

Transactions

Browser path	Screen module
/cash-transactions	../pages/cash-transactions-page
/cash-transactions/new	../pages/new-cash-transaction
/cash-transactions/:id	../pages/edit-cash-transaction
/cash-vouchers	../pages/cash-vouchers-page
/cash-vouchers/new	../pages/new-cash-voucher
/cash-vouchers/:id	../pages/cash-voucher-details
/commercial-papers	../pages/receivable-commercial-papers
/commercial-papers/receivable	../pages/receivable-commercial-papers
/commercial-papers/payable	../pages/payable-commercial-papers
/commercial-papers/receivable/new	../pages/new-receivable-commercial-paper
/commercial-papers/payable/new	../pages/new-payable-commercial-paper
/commercial-papers/:id	../pages/edit-commercial-paper
/commercial-papers/edit/:id	../pages/edit-commercial-paper
/entries	../pages/daily-entries
/entries/new	../pages/new-daily-entry
/entries/:id	../pages/view-entry
/suppliers-invoices	../../suppliers-invoices/invoices-page
/customers-invoices	../../customers-invoices/invoices-page
/batches-invoices	../../batch-invoices/batch-invoices-page
/customers-invoices/new	../../customers-invoices/new-invoice
/suppliers-invoices/new	../../suppliers-invoices/new-invoice
/batches-invoices/new	../../batch-invoices/new-batch-invoice
/invoices/edit/:id	../pages/edit-invoice
/invoices/:id	../pages/edit-invoice

Bank detail subnavigation
The /banks/:id/* route resolves tabs in bank-details.jsx: info (base detail), accounts, cheques, transactions, transfers and reconciliation. /invoices/:id and /invoices/edit/:id both load the shared edit-invoice page.

09 / API endpoint catalog

Complete inventory of resolved Axios method/path references in the current source. The same backend endpoint may appear in multiple frontend modules. Each table identifies its source file; operation names and caller signatures retain the source spelling. Query/body labels describe the actual wrapper arguments, not a complete server schema.

Legend: GET reads; POST creates or performs an action; PUT updates; PATCH partially updates; DELETE removes. {id}, {batchNumber}, {invoiceNumber}, {accountId}, {itemId}, {reconciliationId} and {type} are dynamic path values. For /bank-transactions/{type}, type selects a filter-option collection.

Accounting / Tree / Accouts Tree

src/features/accounting/tree/accouts-tree/api/accounts-tree.js

Verb	Path	Frontend caller / inputs	Request details
GET	`/accounts/tree`	getAccountsTree	No query/body passed by this wrapper
GET	`/accounts/roots`	getAccountRoots params = {}	Query config: { params }
GET	`/accounts/{id}/children`	getAccountChildren id	No query/body passed by this wrapper
GET	`/accounts/{id}`	getAccountById id	No query/body passed by this wrapper
GET	`/accounts/search`	searchAccounts params = {}	Query config: { params }
POST	`/accounts`	createAccount formData	Body: formData
PUT	`/accounts/{id}`	updateAccount { id, body }	Body: body

Accounting / Tree / Cost Tree

src/features/accounting/tree/cost-tree/api/cost-tree.js

Verb	Path	Frontend caller / inputs	Request details
GET	`/cost-centers/tree`	getCostTree	No query/body passed by this wrapper
GET	`/cost-centers/{id}`	getCostById id	No query/body passed by this wrapper
GET	`/cost-centers/search`	searchCostCenters params = {}	Query config: { params }
POST	`/cost-centers`	createCost formData	Body: formData
POST	`/cost-centers`	updateCost { body }	Body: body

Accounting / Tree / Link Account Cost

src/features/accounting/tree/link-account-cost/api/link-api.js

Verb	Path	Frontend caller / inputs	Request details
POST	`/AccountCostCenterLink`	linkAccountToCostCenter body	Body: body

Auth

src/features/auth/api/auh-api.js

Verb	Path	Frontend caller / inputs	Request details
POST	`/Auth/login`	login userData	Body: userData

Banking / Banks

src/features/banking/banks/api/banks.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/Banks	getAllBanks params	Query config: { params: { pageNumber: 1, pageSize: 20, ...params, }, }
GET	/Banks/{id}	getBankById id	No query/body passed by this wrapper
POST	/Banks	createBank payload	Body: payload
PUT	/Banks/{id}	updateBank { id, ...payload }	Body: payload
GET	/BankAccounts	getBankAccounts bankId, params = {}	Query config: { params: { pageNumber: 1, pageSize: 20, bankId, ...params, }, }
GET	/BankAccounts/{id}	getBankAccountById id	No query/body passed by this wrapper
DELETE	/Banks/{id}	deleteBank id	No query/body passed by this wrapper
POST	/BankAccounts	createBankAccount payload	Body: payload
PUT	/BankAccounts/{id}	updateBankAccount { id, ...payload }	Body: payload
DELETE	/BankAccounts/{id}	deleteBankAccount id	No query/body passed by this wrapper
GET	/BankAccounts	getAllBankAccounts params = {}	Query config: { params: { pageNumber: 1, pageSize: 100, ...params }, }
GET	/bank-transactions	getBankTransactions params = {}	Query config: { params }
GET	/bank-transactions/{type}	getBankTransactionFilterOptions type	No query/body passed by this wrapper
GET	/bank-transfers	getBankTransfers params = {}	Query config: { params }
GET	/bank-transfers/{id}	getBankTransferById id	No query/body passed by this wrapper
POST	/bank-transfers/{id}/approve	approveBankTransfer id	Body: none
POST	/bank-transfers/{id}/cancel	cancelBankTransfer { id, cancelReason = '' }	Body: { cancelReason, }
POST	/bank-transfers	createBankTransfer payload	Body: payload

Banking / Cheques

src/features/banking/cheques/api/cheques.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	`/cheques`	getAllCheques params	Query config: { params }
GET	`/cheques/{id}`	getChequeById id	No query/body passed by this wrapper
POST	`/cheques`	createCheque payload	Body: payload
GET	`/batches/{batchNumber}/invoice-summary`	getBatchInvoiceSummary batchNumber	No query/body passed by this wrapper
PUT	`/cheques/{id}`	updateCheque { id, ...payload }	Body: payload
POST	`/cheques/{id}/attachment`	uploadChequeAttachment { id, file }	Body: body Multipart field: Attachment
DELETE	`/cheques/{id}`	deleteCheque id	No query/body passed by this wrapper
GET	`/cheques/statuses`	getChequeStatuses type	Query config: { params: type !== undefined && type !== null ? { type } : {}, }
PATCH	`/cheques/{id}/status`	updateChequeStatus { id, ...payload }	Body: payload
GET	`/Banks`	getChequeBanks	Query config: { params: { pageNumber: 1, pageSize: 20 }, }
GET	`/customers`	getChequeCustomers	Query config: { params: { pageNumber: 1, pageSize: 200 }, }
GET	`/suppliers`	getSupplierList	Query config: { params: { pageNumber: 1, pageSize: 200 }, }
GET	`/currencies`	getCurrencies	No query/body passed by this wrapper

Banking / Reconciliations

src/features/banking/reconciliations/api/bank-reconciliations.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	`/bank-reconciliations`	getBankReconciliations params = {}	Query config: { params: cleanParams({ pageNumber: 1, pageSize: 20, ...params }), }
POST	`/bank-reconciliations`	createBankReconciliation payload	Body: payload
GET	`/bank-reconciliations/{id}`	getBankReconciliationById id	No query/body passed by this wrapper
GET	`/bank-reconciliations/{id}/items`	getReconciliationItems id, params = {}	Query config: { params: cleanParams(params), }
POST	`/bank-reconciliations/{reconciliationId}/items/{itemId}/match`	matchReconciliationItem { reconciliationId, itemId, bankTransactionIDs, // legacy single-ID callers (current match modal) — accepted as fallback bankTransactionID, bankTransactionId, }	Body: { bankTransactionIDs: ids }
POST	`/bank-reconciliations/{reconciliationId}/items/{itemId}/unmatch`	unmatchReconciliationItem { reconciliationId, itemId }	Body: none
POST	`/bank-reconciliations/{reconciliationId}/items/{itemId}/ignore`	ignoreReconciliationItem { reconciliationId, itemId, notes = '' }	Body: { notes }
POST	`/bank-reconciliations/{id}/finalize`	finalizeReconciliation id	Body: none
POST	`/bank-reconciliations/{id}/reconcile`	reconcileLegacy id	Body: none

Master Data / Customers

src/features/master-data/customers/api/customers.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	`/customers`	getCustomers params = {}	Query config: { params }
GET	`/customers/{id}`	getCustomerById id	No query/body passed by this wrapper

Master Data / Suppliers

src/features/master-data/suppliers/api/suppliers.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/suppliers	getSuppliers params = {}	Query config: { params }
GET	/suppliers/{id}	getSupplier id	No query/body passed by this wrapper
GET	/lookups/supplier-statuses	getSupplierStatuses	No query/body passed by this wrapper
GET	/lookups/provider-classes	getProviderClasses	No query/body passed by this wrapper
GET	/lookups/importance-levels	getImportanceLevels	No query/body passed by this wrapper
GET	/lookups/governorates	getGovernorates	No query/body passed by this wrapper
GET	/operation-types	getOperationTypes	No query/body passed by this wrapper
GET	/networks	getNetworks	No query/body passed by this wrapper
GET	/programs	getPrograms	No query/body passed by this wrapper
GET	/payment-statuses	getPaymentStatuses	No query/body passed by this wrapper
PATCH	/suppliers/{id}/finance-info	updateSupplierFinanceInfo { id, ...payload }	Body: payload

Transactions / Cash Transactions

src/features/transactions/cash-transactions/api/cash-transactions.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/cash-transactions/balance	getCashTransactionsBalance params = {}	Query config: { params }
GET	/cash-transactions	getAllCashTransactions params = {}	Query config: { params: { pageNumber: 1, pageSize: 50, sortBy: 'EntryDate', sortDirection: 'DESC', ...params, }, }
GET	/cash-transactions/{id}	getCashTransactionById id	No query/body passed by this wrapper
POST	/cash-transactions	createCashTransaction payload	Body: payload
PUT	/cash-transactions/{id}	updateCashTransaction { id, ...payload }	Body: payload
DELETE	/cash-transactions/{id}	deleteCashTransaction { id, reason }	Query config: { params: { reason }, }

Transactions / Cash Vouchers

src/features/transactions/cash-vouchers/api/cash-vouchers.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/cash-vouchers	getAllCashVouchers params = {}	Query config: { params }
GET	/cash-vouchers/{id}	getCashVoucherById id	No query/body passed by this wrapper
POST	/cash-vouchers	createCashVoucher payload	Body: payload
PUT	/cash-vouchers/{id}	updateCashVoucher { id, ...payload }	Body: payload
GET	/cash-vouchers/statuses	getCashVoucherStatuses	No query/body passed by this wrapper
GET	/cash-vouchers/invoice-details/{invoiceNumber}	getInvoiceForCashVoucher invoiceNumber	No query/body passed by this wrapper

Transactions / Commercial Papers

src/features/transactions/commercial-papers/api/commercial-papers.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/commercial-papers	getAllCommercialPapers params	Query config: { params, }
GET	/commercial-papers/{id}	getCommercialPaperById id	No query/body passed by this wrapper
GET	/Banks	getBanks	Query config: { params: { pageNumber: 1, pageSize: 20, }, }
GET	/currencies	getCurrencies	No query/body passed by this wrapper
GET	/financial-periods	getFinancialPeriods	No query/body passed by this wrapper
POST	/commercial-papers/notes-receivable	createCommercialPaper { paperType, ...payload }	Body: payload
POST	/commercial-papers/notes-payable	createCommercialPaper { paperType, ...payload }	Body: payload
PUT	/commercial-papers/{id}	updateCommercialPaper { id, ...payload }	Body: payload
DELETE	/commercial-papers/{id}	deleteCommercialPaper id	No query/body passed by this wrapper

Transactions / Entries

src/features/transactions/entries/api/entries.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	`/journal-entries`	getJournalEntries params = {}	Query config: { params: requestParams, }
GET	`/journal-entries/{id}`	getJournalEntryById id	No query/body passed by this wrapper
GET	`/journal-entries/statuses`	getJournalEntryStatuses	No query/body passed by this wrapper
POST	`/journal-entries`	createJournalEntry payload	Body: payload
PUT	`/journal-entries/{id}`	updateJournalEntry { id, ...payload }	Body: payload
POST	`/journal-entries/{id}/post`	postJournalEntry { id, postedBy }	Body: { postedBy, }
POST	`/journal-entries/{id}/reverse`	reverseJournalEntry { id, reversedBy }	Body: { reversedBy, }
GET	`/batches/{batchNumber}/summary`	getBatchSummary batchNumber	No query/body passed by this wrapper

Transactions / Invoices / Shared

src/features/transactions/invoices/shared/api/invoices-api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/invoices	getAllInvoices params = {}, type	Query config: { params: requestParams, }
GET	/invoices/{id}	getInvoiceById id	No query/body passed by this wrapper
GET	/invoices/by-number/{invoiceNumber}	getInvoiceByNumber invoiceNumber	No query/body passed by this wrapper
GET	/invoices/statuses	getInvoiceStatuses	No query/body passed by this wrapper
GET	/invoices/next-number	getNextInvoiceNumber	No query/body passed by this wrapper
POST	/invoices	createInvoice formData	Body: formData
GET	/batches/{batchNumber}	getBatchByNumber batchNumber	No query/body passed by this wrapper
POST	/batches/create-invoice	createBatchInvoice formData	Body: formData
POST	/batches/{batchNumber}/close	closeBatch batchNumber	Body: none
PUT	/invoices/{id}	updateInvoice { id, ...body }	Body: body
DELETE	/invoices/{id}	deleteInvoice id	No query/body passed by this wrapper
GET	/invoice-types	getInvoiceTypes	No query/body passed by this wrapper
GET	/customers	getCustomers	No query/body passed by this wrapper
GET	/suppliers	getSuppliers	No query/body passed by this wrapper
GET	/financial-periods	getFinancialPeriods	No query/body passed by this wrapper
GET	/products-services/dropdown	getProductsServices	No query/body passed by this wrapper
POST	/invoices/{id}/pay	payInvoice { id, ...payload }	Body: payload

Named query fields in this module: hasBatchOnly, showOnlyCustomersInvoices, showOnlySuppliersInvoices.

Reports / Accounting / Aging Report

src/features/reports/accounting/aging-report/api/aging-report.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/reports/aging-report	getAgingReport filters = {}	Query config: { params: cleanedParams, }
GET	/reports/aging-report/export-excel	getAgingReportExportExcel params = {}	Query config: { params: cleanedParams, } Returns download Blob
GET	/lookups/provider-classes	getProviderClasses	No query/body passed by this wrapper
GET	/lookups/importance-levels	getImportanceLevels	No query/body passed by this wrapper
GET	/lookups/supplier-statuses	getSupplierStatuses	No query/body passed by this wrapper
GET	/lookups/customer-categories	getCustomerCategories	No query/body passed by this wrapper
GET	/lookups/customer-statuses	getCustomerStatuses	No query/body passed by this wrapper

Reports / Accounting / General Ledger

src/features/reports/accounting/general-ledger/api/general-ledger.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/general-ledger/report	getGeneralLedger filters	Query config: { params: cleanedParams, }
GET	/general-ledger/export-excel	getGeneralLedgerExportExcel filters	Query config: { params: cleanedParams, } Returns download Blob

Reports / Banking / Bank Balances

src/features/reports/banking/bank-balances/api/bank-balances.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/bank-reports/balances	getBankBalances filters = {}	Query config: { params: cleanParams(filters), }
GET	/bank-reports/balances/export-excel	getBankBalancesExportExcel params = {}	Query config: { params: cleanParams(params), } Returns download Blob

Reports / Banking / Bank Cheques

src/features/reports/banking/bank-cheques/api/bank-cheques-summary.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/bank-reports/cheques/summary	getBankChequesSummary filters = {}	Query config: { params: cleanParams(filters), }
GET	/bank-reports/cheques/summary/export-excel	getBankChequesSummaryExportExcel params = {}	Query config: { params: cleanParams(params), } Returns download Blob

Reports / Banking / Bank Cheques

src/features/reports/banking/bank-cheques/api/bank-cheques.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/bank-reports/cheques	getBankChequesReport filters = {}	Query config: { params: cleanParams(filters), }
GET	/bank-reports/cheques/export-excel	getBankChequesReportExportExcel params = {}	Query config: { params: cleanParams(params), } Returns download Blob

Reports / Banking / Bank Reconciliation

src/features/reports/banking/bank-reconciliation/api/bank-reconciliation.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/bank-reports/reconciliation	getBankReconciliation filters = {}	Query config: { params: cleanParams(filters), }
GET	/bank-reports/reconciliation/export-excel	getBankReconciliationExportExcel params = {}	Query config: { params: cleanParams(params), } Returns download Blob

Reports / Banking / Bank Statement

src/features/reports/banking/bank-statement/api/bank-statement.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/bank-reports/statement	getBankStatement filters = {}	Query config: { params: cleanParams(filters), }
GET	/bank-reports/statement/export-excel	getBankStatementExportExcel params = {}	Query config: { params: cleanParams(params), } Returns download Blob

Reports / Banking / Bank Transactions

src/features/reports/banking/bank-transactions/api/bank-transactions.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/bank-reports/transactions	getBankTransactionsReport filters = {}	Query config: { params: cleanParams(filters), }
GET	/bank-reports/transactions/export-excel	getBankTransactionsReportExportExcel params = {}	Query config: { params: cleanParams(params), } Returns download Blob

Reports / Banking / Bank Transfers

src/features/reports/banking/bank-transfers/api/bank-transfers.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/bank-reports/transfers	getBankTransfersReport filters = {}	Query config: { params: cleanParams(filters), }
GET	/bank-reports/transfers/export-excel	getBankTransfersReportExportExcel params = {}	Query config: { params: cleanParams(params), } Returns download Blob

Reports / Banking / Cash Flow

src/features/reports/banking/cash-flow/api/cash-flow.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/bank-reports/cash-flow	getBankCashFlow filters = {}	Query config: { params: cleanParams(filters), }
GET	/bank-reports/cash-flow/export-excel	getBankCashFlowExportExcel params = {}	Query config: { params: cleanParams(params), } Returns download Blob

Reports / Banking / Outstanding Cheques

src/features/reports/banking/outstanding-cheques/api/outstanding-cheques.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/bank-reports/outstanding-cheques	getOutstandingCheques filters = {}	Query config: { params: cleanParams(filters), }
GET	/bank-reports/outstanding-cheques/export-excel	getOutstandingChequesExportExcel params = {}	Query config: { params: cleanParams(params), } Returns download Blob

Reports / Financial Statements / Balance Sheet

src/features/reports/financial-statements/balance-sheet/api/balance-sheet.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	/reports/balance-sheet	getBalanceSheet params = {}	Query config: { params: requestParams, }
GET	/reports/balance-sheet/export-excel	getBalanceSheetExportExcel params = {}	Query config: { params: requestParams, } Returns download Blob

Named query fields in this module: asOfDate, costCenterId, filterByCostCenter, financialPeriodId.

Reports / Financial Statements / Income Statement

src/features/reports/financial-statements/income-statement/api/income-statement.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	`/reports/income-statement`	getIncomeStatement params = {}	Query config: { params: requestParams, }
GET	`/reports/income-statement/export-excel`	getIncomeStatementExportExcel params = {}	Query config: { params: requestParams, } Returns download Blob

Named query fields in this module: costCenterId, financialPeriodId, fromDate, toDate.

Reports / Financial Statements / Trial Balance

src/features/reports/financial-statements/trial-balance/api/trial-balance.api.js

Verb	Path	Frontend caller / inputs	Request details
GET	`/reports/trial-balance/roots`	getTrialBalanceRoots params = {}	Query config: { params: buildParams(params), }
GET	`/reports/trial-balance/{accountId}/children`	getTrialBalanceChildren accountId, params = {}	Query config: { params: buildParams(params) }
GET	`/reports/trial-balance/export-excel`	getTrialBalanceExportExcel params = {}	Query config: { params: buildParams(params), } Returns download Blob

Named query fields in this module: costCenterId, filterByCostCenter, financialPeriodId, fromDate, includeOpeningBalance, toDate.

10 / Form field inventory & rules

These are frontend Zod form fields, not asserted backend DTOs. Forms may include display-only fields, string values and row keys that payload helpers transform or omit. The lists below cover named object schemas under validation folders; the payment schema is added separately because it lives in an invoice utility file.

accountSchema

src/features/accounting/tree/accouts-tree/validation/accounts-validation.js

`accountID`, `accountCode`, `nameAr`, `nameEn`, `parentId`, `accountTypeId`, `lockedInJournal`, `isActive`

costSchema

src/features/accounting/tree/cost-tree/validation/cost-validation.js

`costCenterID`, `ccCode`, `nameAr`, `nameEn`, `parentID`, `isActive`, `isFinal`, `user`

bankAccountSchema

src/features/banking/banks/validation/bank-account.validation.js

`accountNumber`, `branch`, `iban`, `accountNameAr`, `accountNameEn`, `currencyID`, `openingBalance`, `minBalance`, `isActive`, `isDefault`

transferSchema

src/features/banking/banks/validation/bank-transfer.validation.js

`transferType`, `fromAccountID`, `toBankID`, `toAccountID`, `partyType`, `partyID`, `amount`, `notes`

bankSchema

src/features/banking/banks/validation/bank.validation.js

`bankCode`, `bankNameAr`, `bankNameEn`, `swiftCode`, `phone`, `email`, `website`, `addressAr`, `addressEn`, `isActive`

chequeSchema

src/features/banking/cheques/validation/cheque.validation.js

`chequeType`, `customerID`, `supplierID`, `chequeNumber`, `chequeDate`, `receiptDate`, `dueDate`, `amount`, `currencyID`, `exchangeRate`, `bankID`, `bankAccountID`, `bankBranchName`, `cardNumber`, `underDeliveryAccountID`, `collectionAccountID`, `counterAccountID`, `costCenterID`, `invoiceID`, `invoices`, `statusID`, `isNonCashable`, `isBearerOnly`, `hasAttachmentPage`, `beneficiaryName`, `branchName`, `notes`

depositSchema

src/features/banking/cheques/validation/cheque.validation.js

`bankAccountID`, `depositDate`, `depositReference`

collectSchema

src/features/banking/cheques/validation/cheque.validation.js

`collectionDate`

returnSchema

src/features/banking/cheques/validation/cheque.validation.js

`returnReason`, `returnDate`

cashSchema

src/features/banking/cheques/validation/cheque.validation.js

`cashDate`

cashVoucherDetailSchema

src/features/transactions/cash-vouchers/validation/cash-voucher.validation.js

`partyID`, `partyName`, `amount`, `notes`

cashVoucherSchema

src/features/transactions/cash-vouchers/validation/cash-voucher.validation.js

`isReceipt`, `date`, `paymentModeId`, `statusId`, `bankId`, `bankAccountId`, `checkNumber`, `receiptDate`, `dueDate`, `costCenterId`, `invoiceNumber`, `details`

commercialPaperSchema

src/features/transactions/commercial-papers/validation/commercial-paper.validation.js

`paperNumber`, `type`, `amount`

detailSchema

src/features/transactions/entries/validation/journal-entry.validation.js

`rowKey`, `journalEntryDetailID`, `invoiceNumber`, `accountID`, `accountCode`, `accountNameAr`, `costCenterID`, `costCenterCode`, `costCenterNameAr`, `customerID`, `supplierID`, `customerNameAr`, `supplierNameAr`, `recordDate`, `documentNumber`, `debitAmount`, `creditAmount`, `description`

journalEntrySchema

src/features/transactions/entries/validation/journal-entry.validation.js

`entryDate`, `journalType`, `description`, `referenceNumber`, `financialPeriodID`, `statusID`, `currencyID`, `exchangeRate`, `details`

invoiceDetailSchema

src/features/transactions/invoices/shared/validation/invoice.validation.js

`invoiceDetailID`, `productServiceID`, `quantity`, `unitPrice`, `discountPercentage`, `taxPercentage`

invoiceSchema

src/features/transactions/invoices/shared/validation/invoice.validation.js

`invoiceNumber`, `invoiceDate`, `dueDate`, `invoiceTypeID`, `customerID`, `supplierID`, `taxAmount`, `discountAmount`, `financialPeriodID`, `statusId`, `details`

payInvoiceSchema

Source: src/features/transactions/invoices/shared/utils/pay-invoice-utils.js

paymentMode, amount, paymentDate, referenceNumber, bankID, bankAccountID, accountID, checkNumber, fromBankAccountID, toBankAccountID.

Key cross-field rules

Form	Rule
Journal entry	At least one detail; balanced nonzero debit/credit totals; required account on each line.
Invoice	Due date cannot precede invoice date; a customer or supplier must be selected; at least one detail.
Cheque	Received cheques require a customer; issued cheques require a supplier or selected invoices. Receipt/due dates cannot precede cheque date, and due date cannot precede receipt date.
Transfer	Positive amount. Internal transfers need destination bank and account; external transfers need a party.
Payment	Check requires bank, bank account and check number; Cash requires accountID; BankTransfer requires from/to bank accounts.
Batch cheque selection	No duplicate invoice; supplier IDs must agree when available, otherwise supplier names must agree.

11 / Source references & maintenance

This document was generated on 14 October 2026 from the local working tree of mediconsult-finance, with Git baseline 91ba6a8. It includes the project README added in this session and the current implementation of API wrappers, route files, validation schemas and workflow helpers.

Reference	What it establishes
`README.md and package.json`	Project summary, dependencies, scripts and local setup.
`src/index.css and index.html`	Theme colors, typography and RTL rendering.
`src/main.jsx and src/app/`	Runtime providers, routing, authentication layout, HTTP and query configuration.
`src/features/**/api/*.js`	HTTP verbs, paths, wrapper inputs, response mapping and downloads. Per-module source paths are printed throughout the API catalog.
`src/features/**/routes/*.jsx`	Feature UI route declarations and lazy page imports.
`src/features/**/validation/*.js`	Form field sets and validation constraints.
`src/features/**/hooks/`	Query keys, mutations, invalidation and export behavior.
`src/features/**/utils/`	Payload mapping, filters, calculations and batch selection rules.
`vercel.json and vite.config.js`	Static-host route fallback and build plugins.

Keeping this reference accurate

Regenerate the API and route inventories when wrappers or route declarations change. Recheck query defaults and payload helpers when backend contracts evolve. Confirm response DTOs, status codes, authorization rules and state transitions with backend documentation or tests before treating this guide as an integration specification.

Coverage checks
The extraction resolves every axiosInstance HTTP call in the inspected JavaScript/JSX files, including dynamic commercial-paper creation paths. It preserves duplicate callers, all declared path entries, and named object validation schemas. No live backend requests were made and no credentials or private environment values are included.